Machine Translated by Google

列存储与行存储:它们有何不同 真的吗?

丹尼尔·J·阿巴迪
耶鲁大学
美国康涅狄格州纽黑文
dna@cs.yale.edu

塞缪尔·马登
麻省理工学院
美国马萨诸塞州剑桥
madden@csail.mit.edu

纳比尔·哈赫姆
先锋咨询有限公司
美国马萨诸塞州什鲁斯伯里
nhachem@agdba.com

抽象的

最近有很多令人兴奋的事情和工作
在面向列的数据库系统（“列存储”)上。这些
事实证明,数据库系统在分析工作负载（例如在

数据仓库、决策支持和商业智能应用程序。这种性能差异背后的电梯间距是

简单明了:列存储对于只读而言 I/O 效率更高
查询,因为它们只需从磁盘（或内存)读取
通过查询访问的那些属性。

这种简单化的观点导致了这样一种假设:可以使用行存储获得列存储的性能优势:

通过垂直分区模式,或者通过索引每个
列,以便可以独立访问列。在本文中,我们证明这个假设是错误的。我们比较

商业行存储在具有列存储的各种不同配置下的性能,并表明行存储

最近提出的数据的性能明显变慢
仓库基准。然后我们分析性能差异并表明两者之间存在一些重要差异

查询执行器级别的两个系统（除了存储层级别的明显差异）。使用列存储,

然后我们梳理这些差异,展示其影响
各种面向列的查询执行技术的性能,包括矢量化查询处理、压缩和

我们在本文中介绍了新的连接算法。我们的结论是
虽然行存储实现某些功能并非不可能
列存储的性能优势,必须进行更改
到存储层和查询执行器以完全获取
面向列的方法的好处。

类别和主题描述符

H.2.4 [数据库管理]:系统　查询处理,关系数据库

允许将本作品的全部或部分制作为数字或硬拷贝
个人或课堂使用可免费使用,前提是提供副本
并非为了利润或商业利益而制作或分发,并且复制
在第一页上注明此通知和完整引文。否则要复制,
重新发布、发布到服务器或重新分发到列表,需要事先特定
许可和/或费用。
SIGMOD　08,2008 年 6 月 9 日至 12 日,加拿大不列颠哥伦比亚省温哥华。
版权所有 2008 ACM 978-1-60558-102-6/08/06 ...5.00 美元。

一般条款

实验、性能、测量

关键词

C-Store、列存储、面向列的 DBMS、隐形连接、压缩、元组重建、元组物化。

1. 简介

近年来出现了许多面向列的数据库系统,包括 MonetDB [9, 10] 和 C-Store [22]。

这些系统的作者声称他们的方法在某些工作负载上提供了数量级的增益,特别是在读取密集型负载上
分析处理工作负载,例如数据中遇到的工作负载
仓库。

事实上,描述面向列的数据库系统的论文通常包括性能结果,显示相对于传统的、面向行的数据库（商业或开源)的优势。

然而,这些评估通常针对使用“传统”物理设计的面向行的系统进行基准测试,其中包括

具有或多或少一对一关系的面向行的表的集合
映射到逻辑模式中的表。虽然这样的结果
清楚地展示了面向列的方法的潜力,
他们留下了一个关键问题:这些性能提升是否是由于
关于面向列的 DBMS 方式的基本知识
是内部设计的,或者这样的收益也可能在
使用更加面向列的物理结构的传统系统
设计?

通常,基于列的系统的设计者声称从头开始的列存储和使用面向列的物理设计的行存储之间存在根本区别,而没有实际探索行存储系统的替代物理设计。因此,

本文的目标之一就是系统地回答这个问题
方式。本文的作者之一是一位专业 DBA,专门研究流行的商业面向行数据库。他有

为最近提出的数据仓库基准 Star 精心实施了许多不同的物理数据库设计

模式基准 (SSBM) [18, 19],探索如下设计
尽可能“以列为导向”（除了更传统的设计之外）,包括:

- 将系统中的表垂直划分为由（表键、属性)组成的两列表的集合

  对,这样只需要读取必要的列
  回答查询。

- 使用纯指数计划;通过创建索引集合
  覆盖查询中使用的所有列,这是可能的

Machine Translated by Google

让数据库系统无需访问即可回答查询
到底层（面向行)表。

· 使用物化视图的集合,以便有
包含回答每个查询所需的列的视图
在基准测试中。虽然这种方法占用了大量空间，
它是行存储的“最佳情况”,并提供了有用的
与列存储实现的比较点。

我们将这些不同技术的性能与
开源 C-Store 数据库 [22] 的基准性能
SSBM,表明尽管上述方法的能力
为了模拟行存储内列存储的物理结构,它们的查询处理性能相当差。因此,一

这项工作的贡献表明事实上存在一些东西
列存储系统设计的基础,使得
它们更适合数据仓库工作负载。这很重要,因为它打破了一个普遍的说法,即这很容易

现有的面向行的供应商采用面向列的物理数据库设计。我们强调,我们的目标不是找到

在我们的面向行中执行速度最快的 SSBM 实现
数据库,而是评估特定“列”的性能
物理实现,这引出了第二个问题：
文献中提出的许多列数据库特定优化中,哪一个对仓库中列存储相对于行存储的显着性能优势最有影响

工作量?

先前的研究表明重要的优化
面向列的 DBMS 的具体特征包括：

· 后期物化（与块迭代结合使用时）
下面的优化,这种技术也称为向量化查询处理 [9, 25]）,其中列从磁盘读取

在查询中尽可能晚地将行连接在一起
计划[5]。

· 块迭代[25],其中来自列的多个值
作为一个块从一个运算符传递到下一个运算符,而不是
比使用 Volcano 风格的每元组迭代器 [11]。如果值是固定宽度的，
则它们将作为数组进行迭代。

· 特定于列的压缩技术,例如游程长度
编码,在使用后期实现计划时直接对压缩数据进行操作[4]。

· 我们还提出了一种新的优化,称为隐形连接，
这大大提高了后期材料序列化列存储中的连接性能,特别是在模式类型上

在数据仓库中找到。

然而,由于这些技术中的每一种都在
单独的研究论文,没有任何工作能够准确分析其中的哪一个
这些成果最为显着。因此,第三个贡献
这项工作是仔细测量便利店的不同变体
通过逐一删除这些特定于列的优化（实际上,使 C-Store 查询执行器的行为更像

行存储）,分解了其良好性能的因素。我们发现压缩可以提供数量级的

在可能的情况下获得收益,但收益不太显着
其他情况下,而延迟物化提供了大约 3 倍的系数
全面提升性能。其他优化 - 包括块迭代和我们新的隐形连接技术,提供了

平均性能提升 1.5 倍。

总之,我们在本文中做出了三点贡献：

1. 我们展示了尝试在行存储中模拟列存储
不会产生良好的性能结果,并且各种
通常被视为对仓库性能“有利”的技术（仅索引计划、位图索引等)对提高仓库性能几乎没有帮助
改善情况。

2.我们提出了一种提高连接性能的新技术
在列存储中称为隐形连接。我们通过实验证明,在许多情况下,使用此技术执行联接的性能与从已实现联接的单个非规范化表中选择和提取数据一样好,甚至更好。我们因此

得出的结论是,非规范化是一种重要但昂贵的方法
（在空间要求方面)和复杂（在提前决定哪些表要非规范化方面)性能增强

行存储（尤其是数据仓库)中使用的技术是
在列存储中不需要（或者可以与大量使用
降低成本和复杂性）。

3.我们分解列数据库性能的来源
关于仓库工作负载,探索后期物化、压缩、块迭代和隐形的贡献

加入整体系统性能。我们的结果验证了
先前关于新数据的列存储性能的声明
仓储基准（SSBM）,并证明
简单的面向列的操作 – 无需压缩和
后期物化　并没有显着优于优化良好的行存储设计。

本文的其余部分组织如下:我们首先描述面向列的数据库的先前工作,包括调查过去的性能比较并描述一些

为面向列而提出的架构创新
DBMS（第 2 节）;然后,我们回顾 SSBM（第 3 节）。我们
然后描述我们的物理数据库设计技术
面向行的系统（第 4 节）,以及物理布局和查询
C-Store 系统使用的执行技术（第 5 节）。我们
然后展示两个系统之间的性能比较，
首先将我们的行导向设计与基准便利店进行对比
性能,然后将 C-Store 的性能分解为
衡量其用于高效查询执行的哪些技术在 SSBM 上最有效（第 6 节）。

2. 背景和先前的工作

在本节中,我们将简要介绍相关的工作来表征
列存储相对于传统行存储的性能。

尽管垂直分区数据库表的想法
提高性能已经存在很长时间了[1,7,16]，
MonetDB [10] 和 MonetDB/X100 [9] 系统开创了
现代面向列的数据库系统和矢量化查询执行的设计。他们表明,面向列的设计 –

由于卓越的 CPU 和缓存性能（除了减少 I/O 之外） 可以显着优于商业和开放

TPC-H 等基准测试的源数据库。 MonetDB 的工作
然而,并没有试图评估什么样的表现
可以使用面向列的技术从行存储中实现,并且
据我们所知,他们的优化从未被
在与直接的 C-Store 优化相同的上下文中进行评估
对压缩数据的操作。

断裂镜像方法[21]是另一种最近的列存储系统,其中提出了混合行/列方法。

这里,行存储主要处理更新,列存储主要处理读取,后台进程将数据从行存储迁移到列存储。这部作品

Machine Translated by Google

还探索了行存储（Shore)中完全垂直分区策略的几种不同表示,得出的结论是,朴素方案中的元组开销是一个重要问题,并且从磁盘预取大块元组对于改进元组至关重要重建时间。

C-Store [22] 是一种更新的面向列的 DBMS。它包含许多与 MonetDB/X100 相同的功能,以及对压缩数据直接操作的优化 [4]。与其他两个系统一样,它表明列存储在仓库工作负载上可以显着优于行存储,但没有仔细探索可行的行存储物理设计的设计空间。在本文中,我们剖析了 C-Store 的性能,注意文献中提出的各种优化（例如,[4, 5])如何在完整的数据仓库基准测试中对其相对于行存储的整体性能做出贡献,这是便利商店小组之前的工作尚未完成的。

哈里佐普洛斯等人。 [14] 比较从头开始构建的行存储和列存储的性能,研究仅从磁盘扫描数据并立即构建元组的简单计划（“早期物化”）。这项工作表明,在具有简单计划的精心控制的环境中,列存储在其从磁盘读取的列的比例方面优于行存储,但没有专门考虑提高行存储性能的优化,也没有关注某些方面的优化。提高列存储性能的先进技术。

霍尔沃森等人。 [13] 在 Shore 中构建了列存储实现并将 Shore 的未修改（基于行)版本与 Shore 的垂直分区变体进行了比较。他们的工作提出了一种称为“超级元组”的优化,可以避免重复标头信息并将许多元组一起批处理在一个块中,这可以减少完全垂直分区方案的开销,并且对于包含在论文,使垂直分区数据库与列存储竞争。然而,本文并没有探讨许多最近的面向列的优化的性能优势,包括各种不同的压缩方法或后期实现。尽管如此,“超级元组”是一种更高级别的优化类型,本文得出结论,需要将其添加到行存储中以模拟列存储性能。

3. 星型模式基准

在本文中,我们使用星型模式基准（SSBM)[18, 19]来比较便利店和商业商店的性能
行存储。

SSBM 是源自 TPC 的数据仓库基准测试
H [1]。与 TPC-H 不同,它使用纯粹的教科书星型模式（数据仓库的“最佳实践”数据组织）。它还包含比 TPC-H 更少的查询,并且对允许和不允许的调整形式的要求也不太严格。我们选择它是因为它比 TPC-H 更容易实现,而且我们不必修改 C-Store 来让它运行（我们必须这样做才能运行整个 TPC-H 基准测试）。

架构:基准测试由单个事实表（LINE-ORDER 表)组成,它结合了 TPC-H 的 LINEITEM 和 ORDERS 表。这是一个 17 列的表,其中包含有关各个订单的信息,并具有由 ORDERKEY 和 LINENUMBER 属性组成的复合主键。LINEORDER 表中的其他属性包括对 CUSTOMER、PART、SUPP-LIER 和 DATE 表（订单日期和提交日期)的外键引用,以及每个订单的属性,包括其优先级、数量、价格,还有折扣。维度表包含信息以预期的方式了解其各自的实体。图 1（改编自 [19] 的图 2)显示了表的架构。

与 TPC-H 一样,有一个基本“比例因子”,可用于缩放基准的大小。每个表的大小是相对于该比例因子定义的。在本文中,我们使用比例因子 10（生成包含 60,000,000 个元组的 LINEORDER 表）。

图 1:SSBM 基准的架构

查询： SSBM 由 13 个查询组成,分为四个类别,或“航班”：

1. 航班 1 包含 3 个查询。查询对 1 维属性以及 LINEORDER 表的 DISCOUNT 和 QUANTITY 列有限制。查询测量在给定年份中消除不同订单数量的不同级别的折扣时将实现的收入收益（EXTENDEDPRICE 和 DISCOUNT 的乘积）。三个查询的 LINEORDER 选择性为 1.9×10－2 6.5 × 10－4 , , 和 7.5 × 10－5 ,分别。
2. 航班 2 包含 3 个查询。查询对二维属性有限制,并计算特定区域中特定产品类别的收入（按产品类别和年份分组）。三个查询的 LINEORDER 选择性分别为 8.0×10－3 。 , 1.6×10－3, 和 2.0×10－4 ,分别
3. Flight 3由4个查询组成,限制3个维度。查询计算特定区域在一段时间内的收入,按客户国家/地区、供应商国家/地区和年份分组。四个查询的 LINEORDER 选择性分别为 3.4 × 10－2、1.4 × 10－3和 7.6 × 10－7 。 , , 5.5× 10－5,
4. 航班 4 由三个查询组成。查询限制在三个维度列上,并计算按查询 1 的年份、国家和类别分组的利润 (REVENUE - SUPPLY-COST);对于查询 2 和 3,区域和类别。三个查询的 LINEORDER 选择性分别为 1.6×10－2和 9.1 × 10－5 。 , 4.5×10－3,

[1] http://www.tpc.org/tpch/。

Machine Translated by Google

4. 面向行的执行

在本节中,我们将讨论几种不同的技术,这些技术可以
用于在商业中实现列数据库设计
面向行的 DBMS（以下简称 System X）。我们研究三种不同类别的物理设计:完全垂直分区的设计,
“仅索引”设计和物化视图设计。在我们的
评估时,我们还与“标准”行存储设计进行比较
每个关系有一个物理表。

垂直分区:最直接的模拟方法
行存储中的列存储方法是完全垂直划分每个关系[16]。在完全垂直分区的方法中,

需要某种机制来连接同一行中的字段
在一起（列存储通常通过隐式匹配记录
以相同的顺序存储列,但这种优化不是
可在行店中购买）。要实现这一点,最简单的方法
是向每个表添加一个整数“位置”列 - 这通常比使用主键更可取,因为主键可以

很大并且有时是复合的（如 SSBM 中的行顺序表的情况）。这种方法创建一个物理表

逻辑模式中的每一列,其中第 i 个表有两个
列,其中包含逻辑模式第 i 列的值,以及
与位置列中相应的值。查询
然后重写以在位置属性上执行连接
从同一关系中获取多个列。在我们的实现中,默认情况下,System X 选择为此使用散列连接
目的,事实证明这是昂贵的。因此,我们尝试在位置列上添加聚集索引

每个表,并强制 System X 使用索引连接,但这确实
不提高性能 – 索引引起的额外 I/O
访问使它们比散列连接慢。

仅索引计划:垂直分区方法有两种
问题。首先,它要求每一列都存储位置属性,这浪费了空间和磁盘带宽。第二、最

行存储在每个元组上存储相对较大的标头,这
进一步浪费空间（通常是列存储 - 或者甚至可能
根据定义 - 将标题存储在单独的列中以避免这些
间接费用）。为了缓解这些担忧,我们采用第二种方法
考虑使用仅索引计划,其中使用标准的、面向行的设计来存储基本关系,但使用额外的非集群
每个表的每一列都添加B+Tree索引。仅索引
计划　需要数据库的特殊支持,但
由 System X 实现 - 通过构建满足每个表上的谓词的（记录 ID,值)对列表并合并来工作
当有多个谓词时,内存中的这些rid-lists
同一张桌子。当必填字段没有谓词时,所有字段的列表
可以生成列中的（记录 ID,值)对。这样的
计划永远不会访问磁盘上的实际元组。尽管指数仍
显式存储 Rid,它们不存储重复的列值,并且
由于元组标头不存储在索引中,因此它们通常具有比垂直分区方法更低的每元组开销。

仅索引方法的一个问题是,如果一列
没有谓词,仅索引方法需要索引
进行扫描以提取所需的值,这可能会比较慢
比扫描堆文件（如垂直分区方法中发生的情况）。因此,对仅索引方法的优化

是使用复合键创建索引,其中辅助键
来自无谓词列。例如,考虑查询
SELECT AVG(salary) FROM emp WHERE > 40 – 如果我们
有一个带有（年龄,薪水)键的复合索引,那么我们可以直接从该索引回答该查询。如果我们有单独的索引
对于（年龄)和（薪水）,仅索引计划必须找到记录 ID
对应年龄满足的记录,然后合并
包含从中提取的（记录 ID、工资)对的完整列表
（工资)指数,会慢得多。我们通过存储每个的主键来在我们的实现中使用这种优化

维度表作为索引上的辅助排序属性
该维度表的属性。这样我们就可以高效的访问需要连接的维度的主键值

与事实表。

物化视图:我们考虑的第三种方法使用物化视图。在这种方法中,我们为工作负载中的每个查询航班创建一组最佳的物化视图,其中给定航班的最佳视图仅包含回答所需的列

该航班上的查询。我们不会预先加入来自不同的列
这些视图中的表。我们这一策略的目标是让
System X 只从磁盘访问它需要的数据,避免了
显式存储记录 ID 或位置以及存储的开销
每个元组只有一次元组标头。因此,我们期望它能够执行
比其他两种方法更好,尽管它确实需要
查询工作量要提前知道,才实用
在有限的情况下。

5. 面向列的执行

现在我们已经介绍了面向行的设计,在本节中,我们回顾了用于提高面向列的数据库系统性能的三种常见优化,并介绍了

隐形连接。

5.1 压缩

使用面向列的压缩算法压缩数据
并在操作时将数据保存为这种压缩格式
已被证明可以将查询性能提高一个数量级 [4]。直观上,列中存储的数据更多

比存储在行中的数据可压缩。压缩算法
在低信息熵的数据上表现更好（高数据
值局部性）。以包含客户信息（姓名、电话号码、电子邮件地址、

蜗牛邮件地址等）。将数据存储在列中允许所有
姓名存储在一起,所有电话号码存储在一起,
等等。当然,电话号码彼此之间的相似度比
周围的文本字段,例如电子邮件地址或姓名。更远,
如果数据按其中一列排序,则该列将是
超级可压缩（例如,相同值的运行可以是
游程编码）。

但当然,上述观察只会立即影响
压缩率。磁盘空间很便宜,而且越来越便宜
（当然,减少所需磁盘的数量将降低功耗,这是一个变得越来越重要的成本因素）。然而,压缩可以提高性能（在

除了减少磁盘空间之外）,因为如果数据被压缩,那么
当数据从磁盘读取到内存（或从内存读取到 CPU)时,I/O 上花费的时间会更少。因此,一些优化压缩比的“重量级”压缩方案

（例如 Lempel-Ziv、Huffman 或算术编码）,可能是
不如牺牲压缩比以获得减压性能的“轻量级”方案[4, 26]。事实上,压缩可以提高查询性能,而不仅仅是节省时间

输入/输出。如果面向列的查询执行器可以直接操作
压缩数据,完全可以避免解压,
性能可以进一步提高。例如,对于方案
就像游程编码一样　其中重复值的序列是
替换为计数和值（例如,1, 1, 1, 2, 2 → 1×3, 2×2）
– 直接对压缩数据进行操作导致能够
查询执行器对多个列执行相同的操作
一次值,进一步降低 CPU 成本。

先前的工作[4]得出的结论是,两者之间的最大区别

Machine Translated by Google

行存储中的压缩和列存储中的压缩是对列进行排序（或二次排序)并且在列中连续重复相同值的情况。在列存储中,汇总这些值重复并直接对此汇总进行操作非常容易。在行存储中,来自其他属性的周围数据使这个过程变得非常复杂。因此,一般来说,如果查询访问的列中有很大一部分具有某种程度的顺序,则压缩将对查询性能产生更大的影响。对于我们在本文中使用的基准,我们不会以不同的排序顺序存储事实表的多个副本,因此事实表中的十七列中只有一列可以排序（另外两列可以进行二次排序）,因此我们与使用更积极的冗余相比,预计压缩对性能的影响要小一些（每个查询的变量更多）。

5.2 后期物化

在列存储中,有关逻辑实体（例如,一个人)的信息存储在磁盘上的多个位置（例如,姓名、电子邮件地址、电话号码等都存储在单独的列中）,而在行存储此类信息通常位于表的单行中。然而,大多数查询会访问特定实体的多个属性。此外,大多数数据库输出标准（例如,ODBC 和 JDBC)一次访问数据库结果一个实体（而不是一次一个列）。因此,在大多数查询计划中的某个时刻,来自多个列的数据必须组合在一起形成有关实体的信息“行”。因此,这种类似于连接的元组具体化（也称为“元组构造”)是列存储中极其常见的操作。

朴素列存储 [13, 14] 将数据逐列存储在磁盘（或内存中)上,仅读入（从磁盘或内存到 CPU)与特定查询相关的那些列,从其组件属性构造元组,并对这些行执行普通的行存储运算符来处理（例如,选择、聚合和连接)数据。尽管在数据仓库工作负载上可能仍然优于行存储,但这种在查询计划早期构建元组的方法（“早期物化”)使面向列的数据库的大部分性能潜力未能实现。

较新的列存储（例如 X100、C-Store,以及较小程度上的 Sybase IQ)选择将数据保留在列中,直到稍后进入查询计划,直接对这些列进行操作。为此,通常需要构造中间“位置”列表,以便匹配在不同列上执行的操作。例如,一个查询在两列上应用谓词,并从传递谓词的所有元组中投影第三个属性。在使用后期物化的列存储中,谓词分别应用于每个属性的列,并生成通过谓词的值的位置列表（列内的顺序偏移量）。根据谓词选择性,该位置列表可以表示为简单数组、位串（其中第 i 位中的 1 表示第 i 个值通过谓词)或表示为一组位置范围。然后将这些位置表示相交（如果它们是位串,则可以使用按位 AND 运算)以创建单个位置列表。然后将该列表发送到第三列以提取

所需的位置。

晚实现的优点有四重。首先,选择和聚合运算符往往会使某些元组的构造变得不必要（如果执行器在构造元组之前等待足够长的时间,它可能能够完全避免构造它）。其次,如果使用面向列的压缩方法压缩数据,则必须在将值与其他列的值组合之前对其进行解压缩。这删除了

上述直接操作压缩数据的优点。第三,当直接对列数据进行操作时,缓存性能得到提高,因为给定的缓存行不会受到给定操作周围不相关属性的污染（如 PAX [6] 所示）。第四,下一小节中描述的块迭代优化对固定长度属性的性能有更大的影响。在行存储中,如果元组中的任何属性是可变宽度的,则整个元组都是可变宽度的。在后期物化列存储中,可以单独操作固定宽度的列。

5.3 块迭代

为了处理一系列元组,行存储首先迭代
每个元组,然后需要通过元组表示接口从这些元组中提取所需的属性[11]。在许多情况下,例如在 MySQL 中,这会导致一次元组处理,其中有 1-2 个函数调用来从每个操作的元组中提取所需的数据（如果它是一个小表达式或谓词求值)与函数调用相比，-ation 的成本较低)[25]。

最近的工作表明,如果元组块立即可用并在单个运算符调用中进行操作,则可以在行存储中减少元组处理的一些每元组开销 [24, 15],并且这是在 IBM 中实现的DB2 [20]。与行存储中的具体实现相反,在所有列存储（我们知道)中,来自同一列的值块在单个函数调用中发送到运算符。此外,不需要提取属性,并且如果列是固定宽度的,则可以直接将这些值作为数组进行迭代。将数据作为数组进行操作不仅可以最大限度地减少每个元组的开销,而且还可以利用现代 CPU 上的并行性潜力,因为可以使用循环流水线技术 [9]。

5.4 隐形连接

对数据仓库的查询,特别是对使用星型模式建模的数据仓库的查询,通常具有以下结构： 在一个（或多个)维度表上使用选择谓词来限制事实表中的元组集。然后,对受限事实表执行一些聚合,通常按其他维度表属性进行分组。因此,需要为每个选择谓词和每个聚合-聚合分组执行事实表和维度表之间的联接。星型模式基准中的查询 3.1 就是一个很好的例子。

选择 c.nation、s.nation、d.year、
sum(lo.revenue) 作为来自客户 AS c、lineorder
AS lo、供应商 AS s、dwdate AS d 的收入

其中 lo.custkey = c.custkey AND lo.suppkey =
s.suppkey AND lo.orderdate = d.datekey AND
c.region = ASIA AND s.region = ASIA AND
d.year >= 1992 且 d.年份 <= 1997

按 c.nation、s.nation、d.year 分组 按 d.year asc、收入 desc;

此查询查找 1992 年至 1997 年间居住在亚洲并购买亚洲供应商提供的产品的客户的总收入,并按客户所在国家/地区、供应商所在国家/地区和国家/地区的每个唯一组合进行分组。交易年份。

执行这些类型查询的传统计划是按照谓词选择性的顺序管道连接。例如,如果 c.region = ASIA 是最具选择性的谓词,则 lineorder 和 customer 表之间 custkey 上的联接为

Machine Translated by Google

首先执行,过滤 lineorder 表,以便仅保留来自亚洲客户的订单。执行此连接时,这些客户的国家/地区将添加到连接中

客户订单表。这些结果被管道化到连接中
与供应商表,其中应用 s.region = ASIA 谓词并提取 s.nation,然后与

数据表和应用的年份谓词。这些结果
然后对连接进行分组和聚合,并根据 ORDER BY 子句对结果进行排序。

传统计划的替代方案是后期物化连接
技术[5]。在这种情况下,谓词应用于 c.region
列 (c.region = ASIA),以及客户密钥
客户表在与该谓词匹配的位置处提取。然后将这些密钥与客户密钥列连接起来

从事实表。该连接的结果是两组位置,一组用于事实表,一组用于维度表,指示各个表中的哪些元组对通过了

连接谓词并被连接。一般情况下,最多选择这两种
位置列表按排序顺序生成（外表在
连接,通常是事实表）。 c.nation 列中的值
然后提取这组（无序)位置,以及
来自另一个事实表的值（使用有序的位置集）
列（供应商代码、订单日期和收入）。类似的连接有
然后使用供应商和日期表执行。

这些计划中的每一个都有一系列缺点。在第一种（传统)情况下,在连接之前构造元组会排除所有
第 5.2 节中描述的后期实现效益。在里面
第二种情况,维度表分组列中的值需要
以错位顺序提取,这可能会产生重大影响
成本[5]。

作为这些查询计划的替代方案,我们引入了一种技术
我们称之为可在面向列的数据库中使用的隐形联接
用于星型模式表上的外键/主键连接。这是
后期物化连接,但最小化需要的值
无序提取,从而减轻这两方面的缺点
如上所述。它的工作原理是将连接重写为谓词
事实表中的外键列。这些谓词可以
通过使用哈希查找来评估（在这种情况下,哈希
join 是模拟的）,或者使用更高级的方法,例如
我们称之为谓词间重写的技术,将在下面第 5.4.2 节中讨论。

通过将连接重写为事实表列上的选择谓词,
它们可以与应用于事实表的其他选择谓词同时执行,并且之前的工作 [5] 中描述的任何
谓词应用算法都可以

用过的。例如,每个谓词可以并行应用,并且
使用快速位图操作将结果合并在一起。或者,谓词应用程序的结果可以被管道化到

另一个谓词应用程序来减少
必须应用第二个谓词。仅当所有谓词都满足后
已应用的是从相关中提取的适当元组
尺寸（这也可以并行完成）。等到
在执行此提取之前已应用所有谓词,
无序提取的数量被最小化。

不可见连接扩展了之前的工作,通过利用面向列的布局来提高星型模式连接 [17, 23]
的性能,这让人想起半连接 [8],并且

重写谓词以避免哈希查找,如下所述。

5.4.1 加入详情

不可见连接分三个阶段执行连接。首先,每个
谓词应用于适当的维度表以提取
满足谓词的维度表键列表。这些键

用于构建一个哈希表,可用于测试是否存在
特定的键值满足谓词（哈希表应该
很容易装入内存,因为维度表通常很小并且
该表仅包含键）。执行此操作的示例
上述对某些示例数据的查询的第一阶段显示在
图 2.

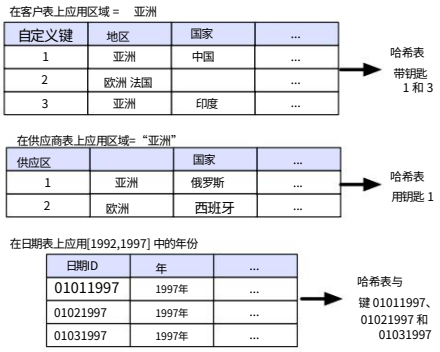


图 2:执行查询所需连接的第一阶段
3.1 来自一些样本数据的星型模式基准测试

在下一阶段,使用每个哈希表来提取位置
事实表中满足相应谓词的记录数。
这是通过探测哈希表中的每个值来完成的
事实表的外键列,创建外键列中满足谓词的所有位置的列表。然后,

所有谓词的位置列表相交以生成
事实表中满足位置 P 的列表。的一个例子
第二阶段的执行如图 3 所示。请注意
位置列表可以是位置的显式列表,也可以是位图
如示例所示。

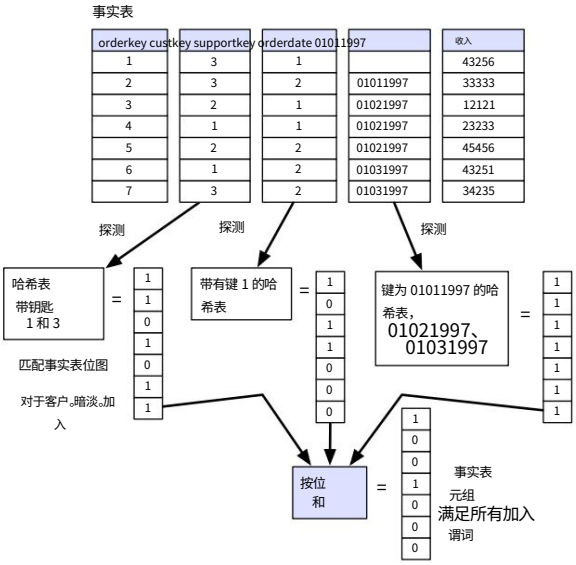


图 3:需要执行的连接的第二阶段
来自某些样本的星型模式基准的查询 3.1
数据

连接的第三阶段使用满足位置 P 的列表
在事实表中。对于事实表中的每一列 C 包含
外键引用需要回答的维度表

Machine Translated by Google

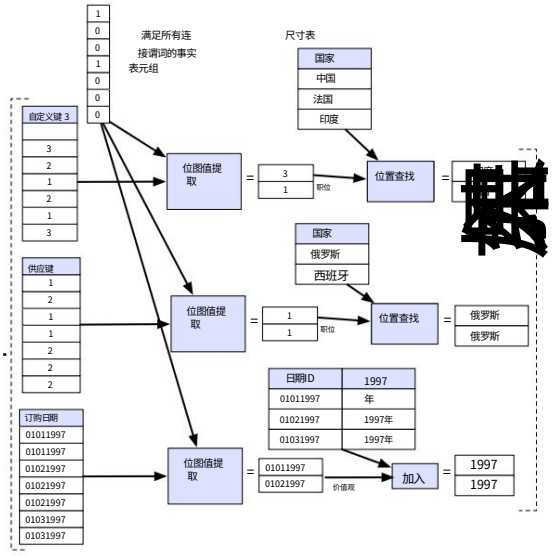


图 4:对某些示例数据执行星型模式基准测试中的查询 3.1 所需的连接的第三阶段

在查询中（例如,在选择列表、分组依据或聚合子句中引用维度列的情况下）,使用 P 提取 C 中的外键值,并在相应的维度表中查找。请注意,如果维度表键是从 1 开始的有序连续标识符列表（这是常见情况）,则外键实际上表示维度表中所需元组的位置。这意味着可以使用此位置列表直接提取所需的维度表列（这只是快速数组查找）。

这种直接数组提取是这种连接不会遭受先前发布的后期材料化的上述陷阱的原因（加上维度表通常很小,因此要查找的列通常可以放入 L2 缓存内）连接方法[5],由于维度表值提取的无序性质,最终位置列表提取的成本非常昂贵。此外,需要提取的数值被最小化,因为 P 中的位置数量取决于整个查询的选择性,而不是仅取决于迄今为止已执行的查询的部分的选择性。

图 4 显示了执行第三阶段的示例。请注意,对于日期表,键列不是从 1 开始的有序、连续的标识符列表,因此必须执行完全连接（而不仅仅是位置提取）。此外,请注意,由于这是一个外键主键连接,并且由于所有谓词都已应用,因此从事实来看,对于相交位置列表中的每个位置,每个维度表中保证只有一个结果桌子。这意味着第三阶段中的每个维度表连接都有相同数量的结果,因此每个连接都可以单独完成,并在查询计划的稍后位置组合（缝合在一起)结果。

5.4.2 谓词间重写正如到目前为止所描述的,该算法

与考虑面向列的半连接或后期物化散列连接的另一种方式没有什么不同。尽管连接的散列部分被表示为事实表列上的谓词,但实际上应用谓词的方式与执行（后期实现)散列连接的方式之间几乎没有什么区别。优势将连接表达为谓词的方法在令人惊讶的常见情况下发挥作用（对于星型模式连接）,其中应用谓词后维度表中保留的键集是连续的。在这种情况下,可以使用一种我们称之为“谓词之间重写”的技术,其中谓词可以从事实表上的哈希查找谓词重写十次为外键落在之间的“之间”谓词。键范围的两端。例如,如果应用谓词后有效的连续键集是键 1000-2000,则不必将这些键中的每一个插入到散列表中并探测散列表中的每个外键值。事实表中,我们可以简单地检查外键是否在 1000 到 2000 之间。如果是,则元组连接;否则它不会。由于显而易见的原因,谓词之间的执行速度更快,因为它们可以直接评估而无需查找任何内容。

应用此优化的能力取决于这些有效维度表键的集合是否连续。在许多情况下,这个属性并不成立。例如,未排序字段上的范围谓词会导致不连续的结果位置。即使对于排序字段上的谓词,按该属性对维度表进行排序的过程也可能对主键进行重新排序,因此它们不再是有序的、连续的标识符集。然而,后一个问题可以通过使用字典编码来重新分配键（而不是压缩)来轻松缓解。由于键是唯一的,所以对列进行字典编码会导致字典键成为从 0 开始的有序连续列表。只要使用相同的字典表对事实表外键列进行编码,哈希表就可以在- 可以执行谓词重写。

此外,关于优化仅适用于维度表的排序列上的谓词的断言并不完全正确。

事实上,数据仓库中的维度表通常包含粒度越来越细的属性集。例如,SSBM 中的日期表具有年份列、年月列和完整日期列。如果表按年排序,其次按年月排序,第三按完成日期排序,则这三列中任何一列上的等式谓词将产生一组连续的结果（或已排序的结果上的范围谓词)柱子）。再例如,供应商表有一个地区列、一个国家列和一个城市列（一个地区有许多国家,一个国家有许多城市）。同样,从左到右排序将导致对这三列中任何一列的谓词产生连续的范围输出。由于 OLAP 做法是在连续查询中汇总数据（按地区告诉我利润,按国家告诉我利润,按城市告诉我利润）,数据仓库查询经常访问这些列。因此,“谓词间重写”的使用频率比人们最初预期的要高,并且（正如我们在下一节中所示)通常会带来显着的性能增益。

请注意,谓词重写不需要更改查询优化器来检测何时可以使用此优化。针对维度表评估谓词的代码能够检测结果集是否连续。如果是,则事实表谓词在运行时被重写。

6. 实验

在本节中,我们将比较面向行的方法在 SSBM 上的 C-Store 性能,目的是回答四个关键问题:

1. 在行存储中模拟列存储的不同尝试与 C-Store 的基准性能相比如何?

Machine Translated by Google

2. 未经修改的行存储是否有可能获得好处？
   适合列导向设计吗？
3. 针对列存储提出的具体优化（压缩、后期物化和块处理），其中
   最重要的是？
4. 使用不可见连接技术在列存储中执行星型模式连接的成本与在非规范化事实表上
   执行查询（其中连接
   已预先执行？

通过回答这些问题，我们为数据库实现者提供了
有兴趣采用面向列的方法
性能优化最富有成果的指南。此外，答案将帮助我们了解需要什么改变

如果行存储要成功模拟列存储，则要在存储管理器和查询执行器级别对行存储进行操作。

我们所有的实验都在 2.8 GHz 单处理器上运行，
双核 Pentium(R) D 工作站，运行 3 GB RAM
RedHat Enterprise Linux 5。该机器有一个 4 磁盘阵列，作为单个逻辑卷进行管理，文件在其中条带化。典型的
对于条带文件，I/O 吞吐量为 40 - 50 MB/秒/磁盘，或者在 ag-gregate 中为
160 - 200 MB/秒。我们报告的数字是平均值
多次运行，并且基于“热”缓冲池（实际上，我们
发现这为两者带来了约 30% 的性能提升
系统；收益并不是特别显着，因为金额
每个查询读取的数据超过缓冲池的大小）。

6.1 实验设置的动机

图 5 比较了 C-Store 和 System X 的性能
星型模式基准。我们提醒读者不要阅读
两者之间的绝对性能差异太多
系统 正如我们在本节中讨论的，除了基本的系统之外，这些系统的实现存在很大的差异

影响这些性能数字的行与列的差异。

在此图中，“RS”指的是基本 System X 案例的编号，
“CS”是指基本便利店案例的编号，“RS (MV)”
指的是系统 X 上的数字，使用物化视图的最佳集合，其中包含所需表的最小投影

回答每个问题（参见第 4 节）。如图所示，便利店表现优于其他
系统 X 在基本情况下增加了六倍，在基础情况下增加了三倍
当系统 X 使用物化视图时。这与
之前的工作表明，列存储在数据仓库工作负载上的性能可以显着优于行存储 [2,9,22]。

然而，图 5 中显示的第四组数字“CS
(Row-MV)”说明了在比较跨系统的数字时需要注意的事项。对于这些数字，我们存储了

C-Store 内的相同（面向行！）物化视图数据。
人们可能认为 C-Store 存储管理器无法存储
行中的数据，因为毕竟它是列存储。然而，这可以
通过使用具有单列类型的表可以轻松完成
“细绳”。此列中的值是整个元组。一个可能
还期望 C-Store 查询执行器无法对行进行操作，因为它需要单独的列作为输入。然而，行
是 C-Store 中的合法中间表示形式 如

在第 5.2 节中解释，在查询计划中的某个时刻，C-Store 从组件列重新构造行（因为用户界面
对于 RDBMS 来说是逐行的）。执行此元组重建后，它继续使用以下命令执行查询计划的其余部分
标准行存储运算符 [5]。因此，“CS (Row-MV)”
“RS（MV）”对以相同方式存储的相同输入数据执行相同的查询。因此，人们可能会期望
这些数字是相同的。

与此预期相反，System X 的速度比 C-Store 的速度快得多（超过两倍）。回想起来，这并不令人惊讶 System X 已经

致力于寻求和消除绩效的团队
代码中存在瓶颈，而 C-Store 有多个已知的性能瓶颈尚未解决 [3]。此外，C-Store 作为一个简单的原型，尚未实现 System X 中可用的高级性能功能。其中两个功能

是分区和多线程。系统 X 能够分区
每个物化视图都针对其设计的查询飞行进行了优化。分区可提高运行时的性能

通过减少需要扫描以回答查询的数据来实现单机化。例如，物化视图用于

查询航班 1 按订单日期年份进行分区，这很有用，因为
此航班中的每个查询都有一个关于 orderdate 的谓词。确定
System X 从分区中获得的性能优势，
我们在相同的物化视图上运行相同的基准，而不对它们进行分区。我们发现平均查询时间在
案例为20.25秒。因此，分区为系统 X 提供了两个优势（尽管这因查询而异，这将是

第 6.2 节进一步讨论）。 C-Store 也有一个缺点，因为它不是多线程的，因此无法接受

额外核心的优势。

因此，我们在本文中试验的两个系统之间存在许多差异。有些是列存储和行存储之间的根本区别，有些是实现

文物。因为很难得出有用的结论
比较不同系统的数字，我们选择不同的
我们的实验设置中的策略，探索基准性能
从两个角度。在 6.2 节中，我们尝试在行存储中模拟列存储。本节的实验仅

在系统 X 上，因此我们不会遇到跨系统比较
问题。在 6.3 节中，我们删除了性能优化
从 C 存储直到实现行存储性能。再说一遍，所有
实验仅在单个系统（C-Store)上进行。

通过以这种方式进行实验，我们能够得出
在不依赖跨系统比较的情况下得出有关列存储性能优势的一些结论。例如，

有趣的是，图 5 中存在不止一个因素
尽管“CS”和“CS（行MV）”之间有六个差异
事实上，它们在同一系统上运行并且都读取最小的
回答每个查询所需的磁盘外列集。显然
列存储的性能优势不仅仅在于 I/O
从磁盘读取数据较少的优点。我们将解释
造成这种性能差异的原因请参阅第 6.3 节。

6.2 行存储中的列存储模拟

在本节中，我们将描述 System X 不同配置在星型架构基准上的性能。我们将
System X 配置为按年份对 lineorder 表进行分区（这意味着为数据库中每年的元组
创建不同的物理分区）。如中所述

6.1 节，这种分区大大加快了 SSBM 查询的速度
涉及 orderdate 谓词（查询 1.1、1.2、1.3、3.4、
4.2、4.3查询只需1年；查询 3.1、3.2 和 3.3 包括
半年内的选择性查询大大减少）。很遗憾，
对于面向列的表示，System X 不允许我们
在 orderdate 上对两列垂直分区进行分区（因为
它们不包含 orderdate 列，当然，
对于 orderdate 垂直分区），这意味着对于那些
限制 orderdate 列的查询航班，面向列的方法相对于基本情况处于劣势。

尽管如此，我们决定对基本情况使用分区

Machine Translated by Google

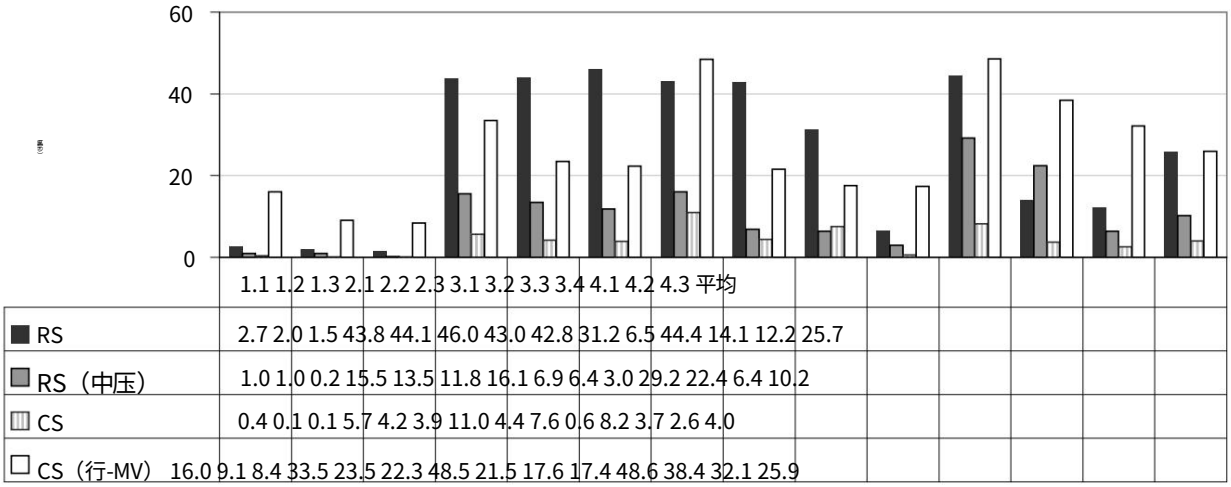


图 5:便利店“CS”和 System X“RS”的基准性能与相同系统上的物化视图案例进行比较。

因为这实际上是数据库管理员在尝试提高行存储上的这些查询的性能时会使用的策略。当我们在没有分区的情况下运行基本案例时,性能平均降低了两倍（尽管每个查询的性能因 orderdate 列上谓词的选择性而异）。因此,如果可以基于两个间接级别对表进行分区（从主键或记录 ID,我们得到 orderdate,并且从订单日期我们得到年份）。

System X 的其他相关配置参数包括:32 KB 磁盘页、用于排序、联接、中间结果的 1.5 GB 最大内存以及 500 MB 缓冲池。我们尝试了不同的缓冲池大小,发现不同的大小不会在查询时间上产生很大的差异（由于在此基准测试中主要使用大表扫描）,除非使用非常小的缓冲池。我们启用了压缩和顺序扫描预取,我们注意到这两种技术都提高了性能,这也是由于处理这些查询需要大量 I/O。System X 还实现了星型连接,并且当优化器期望这将提高查询性能时,它将使用布隆过滤器。

回想一下第 4 节,我们尝试了六种配置 - System X 在 SSBM 上的配置：

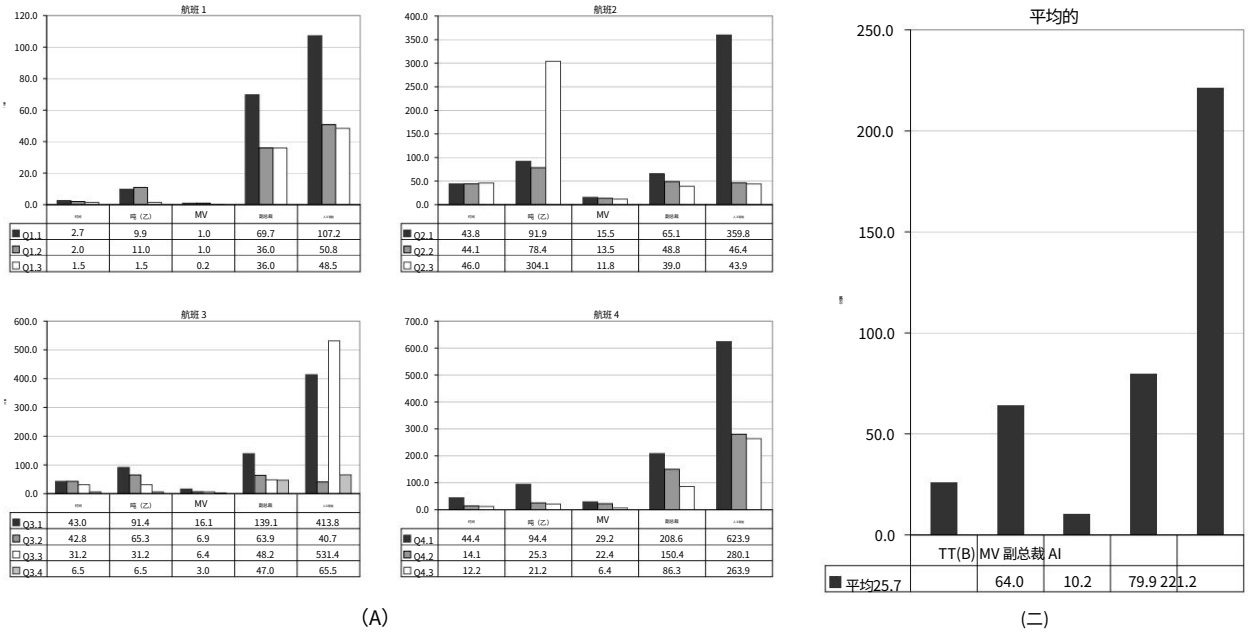
1. “传统”的面向行的表示;在这里,我们允许 System X 使用位图和布隆过滤器（如果它们是有益的）。
2. “传统（位图）”方法,与传统方法类似,但计划偏向使用位图,有时导致它们产生比纯传统方法差的计划。
3. “垂直分区”方法,每一列与原始关系中的记录 ID 都有自己的关系。
4. “仅索引”表示,在面向行的方法中的每一列上使用非聚集B+树,然后通过直接从索引读取值来进行应答查询。
5. “物化视图”方法,为每个查询提供最佳的物化视图集合（在这些视图中没有提前执行任何连接）。

按查询航班细分的详细结果显示在图 6(a),所有查询的平均结果如图所示是6(b)。物化视图在所有情况下都表现最佳,因为它们读取处理查询所需的最少量的数据。在物化视图之后,传统方法或带有位图索引的传统方法通常是最佳选择。平均而言,传统方法比我们模拟面向列的方法的最佳尝试要好大约三倍。

如上所述,对于可以利用 orderdate 分区的查询尤其如此。对于查询航班 2（不受益于分区）,垂直分区方法与传统方法相比具有竞争力;由于我们下面讨论的原因,仅索引方法表现不佳。在更详细地研究各个查询的性能之前,我们总结了限制列式方法方法的两个高级问题:元组开销和低效的元组重建： 元组开销:正如其他人所观察到的[16] ,一行存储中完全垂直分区方法的问题之一是元组开销可能非常大。由于记录 ID 或主键与每列一起存储以允许重建元组的要求进一步加剧了这种情况。我们将垂直分区方法中的列表大小与传统行存储表的大小进行了比较,发现 SSBM 规模 10 行顺序表（包含 6000 万个元组)中的单个列表需要 0.7 到 1.1 GB压缩后存储的数据量 – 这表示每行约 8 字节的开销,再加上 record-id 和列属性各约 4 字节,具体取决于列和压缩有效的程度（16 字节 × 6 × 107元组 = 960 MB）。相比之下,传统方法中的整个 17 列行顺序表解压后占用约 6 GB,压缩后占用 4 GB,这意味着在垂直分区方法中仅扫描其中四列所花费的时间与扫描整个事实表所花费的时间一样长。传统方法。作为比较,在 C-Store 中,单列整数仅需要 240 MB（4 字节 × 6 × 107 元组 = 240 MB）,压缩后的整个表需要 2.3 GB。

列连接:正如我们上面提到的,将同一个表中的两列合并在一起需要连接操作。System X 倾向于使用散列连接来执行这些操作。我们尝试强制 System X 使用索引嵌套循环和合并连接,但发现这并没有提高性能,因为索引访问的开销很高,而且 System X 无法跳过合并连接之前的排序。

Machine Translated by Google



(A) (二)

图 6:(a) 按查询飞行的行存储不同变体的性能数据。这里,T是传统的,T(B)是传统的（位图）,MV是物化视图,VP是垂直分区,AI是所有索引。 (b) 所有查询的平均性能。

6.2.1 详细的行存储性能分解

在本节中,我们使用 System X 为查询 2.1 生成的计划来查看行存储方法的性能

SSBM 作为指导（我们选择这个查询是因为它是
很少有不受益于订单日期分区的,因此提供了传统和垂直之间更平等的比较

分区方法。)虽然我们不剖析其他计划
仔细查询,它们的基本结构是一样的。 SQL 为
这个查询是：

```
SELECT sum(lo.revenue), d.year, p.brand1
FROM lineorder AS lo, dwdate AS d,
        零件 AS p,供应商 AS s
WHERE lo.orderdate = d.datekey
    AND lo.partkey = p.partkey
    AND lo.suppkey = s.suppkey
    AND p.category =  MFGR#12
    AND s.region =  美洲
按 d.year 分组,p.brand1
按 d.year、p.brand1 排序
```

该查询的选择性为 8.0 × 10－3 。在这里,垂直分区方法的性能与传统方法差不多

（65 秒与 43 秒）,但仅索引方法的性能要差得多（360 秒）。我们看看原因

为此。

传统:对于此查询,传统方法扫描整个 lineorder 表,使用散列连接将其与 dwdate 连接,

零件和供应商表（按顺序）。然后,它执行基于排序的聚合来计算最终答案。成本占主导地位
到扫描 lineorder 表的时间,在我们的系统中大约需要 40 秒。物化视图只需 15 秒,

因为他们必须读取传统方法大约 1/3 的数据
方法。
垂直分区:垂直分区方法将partkey列与过滤后的part表进行散列连接,

带有过滤后的供应商表的suppkey列,然后
散列连接这两个结果集。这会生成包含事实表中的 record-id 和部件的 p.brand1 属性的元组
满足查询的表。然后系统 X 将其与
dwdate 表来获取 d.year,最后使用一个额外的
散列连接从其列表中选取 lo.revenue 列。此方法需要 lineorder 表的四列

全文（按顺序)阅读,正如我们上面所说,
需要从磁盘读取与传统方法一样多的字节
方法,并且此扫描成本主导此查询的运行时间,
与传统的相比产生可比的性能
方法。在这种情况下,散列连接会使性能降低大约
25%;我们尝试通过添加来消除散列连接
每个垂直分区中关键列上的聚集B+树,但是
在这种情况下,System X 仍然选择使用散列连接。
仅索引计划:仅索引计划通过以下方式访问所有列
非聚集 B+Tree 索引,在 record-id 上连接来自同一个表的列（因此它们永远不会跟随指针回到基址
关系）。查询 2.1 的计划对
的suppkey、revenue、partkey和orderdate列
事实表,使用散列连接按顺序连接它们。在这个
在这种情况下,索引扫描是整个索引文件的相对快速的顺序扫描,并且不需要在叶页之间查找。这
然而,散列连接非常慢,因为它们组合了两个 6000 万个元组列,每个列占用数百兆字节

的空间。请注意,散列连接可能是这些的最佳选择
连接,因为索引扫描的输出未按记录 ID 排序,并且
对记录 ID 列表进行排序或执行索引嵌套循环可能会
慢得多。正如我们下面讨论的,我们找不到一种方法
强制 System X 将这些连接推迟到计划的后期,这
会使这种方法的性能更接近垂直分区。

连接事实表的列后,计划使用索引
范围扫描提取过滤后的part.category列和
哈希将其与part.brand1列和part.part-连接起来

Machine Translated by Google

键列（均通过完整索引扫描访问）。然后它散列
将此结果与事实表中已连接的列连接起来。
接下来,它散列连接supplier.region（通过索引范围扫描过滤)和supplier.suppkey列（访问
通过全索引扫描）,然后哈希将其与事实表连接起来。最后,它使用完整索引扫描来访问 dwdate.datekey
和 dwdate.year 列,使用散列连接将它们连接起来,然后散列
将结果与事实表连接起来。

6.2.2 讨论

之前的结果表明,我们没有尝试模仿
行存储中的列存储特别有效。垂直的
分区方法可以提供具有竞争力的性能
当只选择几个时,与行存储相同或稍好一些
列。然而,当选择超过大约 1/4 的列时,由于元组标题和冗余副本而浪费的空间

record-id 的性能比传统方法差。
这种方法还需要相对昂贵的散列连接来将事实表中的列组合在一起。有可能是系统

X 可能会被欺骗将列按排序顺序存储在磁盘上
然后使用合并联接（不进行排序)来组合列
来自事实表,但我们的 DBA 无法诱导这种行为
从系统中。
仅索引计划每条记录的开销较低,但引入
另一个问题　即系统被迫连接
在过滤之前使用昂贵的哈希连接将事实表组合在一起
使用维度列的事实表。看来系统 X 是
无法将这些连接推迟到计划的后期（如垂直分区方法所做的那样）,因为它无法保留来自
与另一个表连接后的事实表。这些巨人
散列连接会导致性能极其缓慢。
相对于传统的规划,物化视图是一种
明显的胜利,因为它们允许系统 X 仅读取
相关的事实表,而不将列合并在一起。
位图索引有时会有所帮助 – 特别是当选择性
查询的数量很少　因为它们允许系统跳过
扫描事实表时的某些页面。在其他情况下,他们
降低系统速度,因为合并位图会增加一些开销
计划执行和位图扫描可能比纯顺序慢
扫描。
最后一点,我们发现在 X 系统中实施这些计划是相当痛苦的。我们被要求重写所有的

使用垂直分区方法的查询,并且必须进行
广泛使用优化器提示和其他技巧来诱使系统执行我们想要的操作。

在下一节中,我们将研究列存储系统是如何设计的
从头开始能够规避这些限制,并且
分解不同功能的性能优势
SSBM 基准上的 C-Store 系统。

6.3 列存储性能

检查平均值后立即显而易见
SSBM 上 C-Store 的查询时间（大约 4 秒）
不仅比模拟的面向列的存储更快
行存储（80 秒到 220 秒）,但甚至比
查询已知的行存储的最佳情况
前进,行存储已经创建了定制的物化视图
查询计划（10.2 秒）。无需进一步实验即可立即解释部分性能差异

– 列存储不会受到元组开销和高开销的影响
行存储所做的列连接成本（这将在第 6.3.1 节中解释）。然而,这一观察并没有解释原因
为什么列存储比物化视图案例更快或者
第 6.1 节中的“CS Row-MV”案例,其中
跨系统的I/O类似,其他系统不需要
将同一个表中的列连接在一起。为了理解
针对后者的性能差异,我们在列存储中执行额外的实验,其中我们连续删除面向列的优化,直到列存储开始模拟

行存储。通过这样做,我们了解了这些不同的优化对查询性能的影响。这些结果呈现在

第 6.3.2 节。

6.3.1 元组开销和连接成本

现代列存储不显式存储记录 ID（或
主键)需要将同一表中的列连接在一起。
相反,他们使用隐式列位置来重建列
（每列的第 i 个值属于表中的第 i 个元组）。此外,元组标题存储在它们自己的单独列中

因此它们可以与实际的列值分开访问。因此,列存储中的列仅包含数据

来自该列,而不是垂直分区行存储中的元组标题、记录 ID 和列数据。

在列存储中,堆文件按位置顺序存储（
第 i 个值始终位于第 i − 第一个值之后）,而顺序
许多行存储中的堆文件,即使在集群属性上,也只是
通过索引来保证。这使得合并连接（没有
a sort)是列存储中元组重建的明显选择。在行存储中,由于迭代排序的文件必须是

通过索引间接完成,这可能会导致额外的搜索
在索引叶之间,基于索引的合并连接是一种缓慢的方法
重建元组。
应该指出的是,上述两者之间的差异都不是
列存储性能和行存储性能是基础。行存储没有理由不能存储元组

单独的标头,使用虚拟记录 ID 来连接数据,并按保证的位置顺序维护堆文件。上述观察

只是强调了一些重要的设计考虑因素
如果想要构建一个可以成功的行存储,那么就具有相关性
模拟列存储。

6.3.2 列存储优势细分

如第 5 节所述,三个面向列的优化,
文献中分别提出了这些方法,所有这些都声称可以显着提高列式数据库的性能。这些优化是压缩、后期实现和块迭代。

此外,我们使用隐形连接技术扩展了 C-Store,
我们还期望能够提高性能。据推测,这些优化是造成性能差异的原因

列存储和面向行的物化视图案例来自
图 5（在 System X 和 C-Store 中)具有类似的 I/O
模式作为列存储。为了验证这个推测,
我们陆续从 C-Store 中删除了这些优化
每个步骤后测量性能。
自 C-Store 以来,从 C-Store 中删除压缩很简单
包括一个用于执行此操作的运行时标志。删除不可见的连接
也很简单,因为它是我们自己添加的新运算符。
为了消除延迟实现,我们必须手动代码查询
计划在查询计划的开头构造元组。删除块迭代比其他三个要困难一些

优化。 C-Store 数据“块”可以通过以下方式访问
两个接口:“getNext”和“asArray”。前一种方法需要对每个迭代的值进行一次函数调用,而后者
方法返回一个指向可以直接迭代的数组的指针。对于 SSBM 查询计划中使用的运算符来访问
通过“asArray”接口块,我们编写了替代版本-

Machine Translated by Google

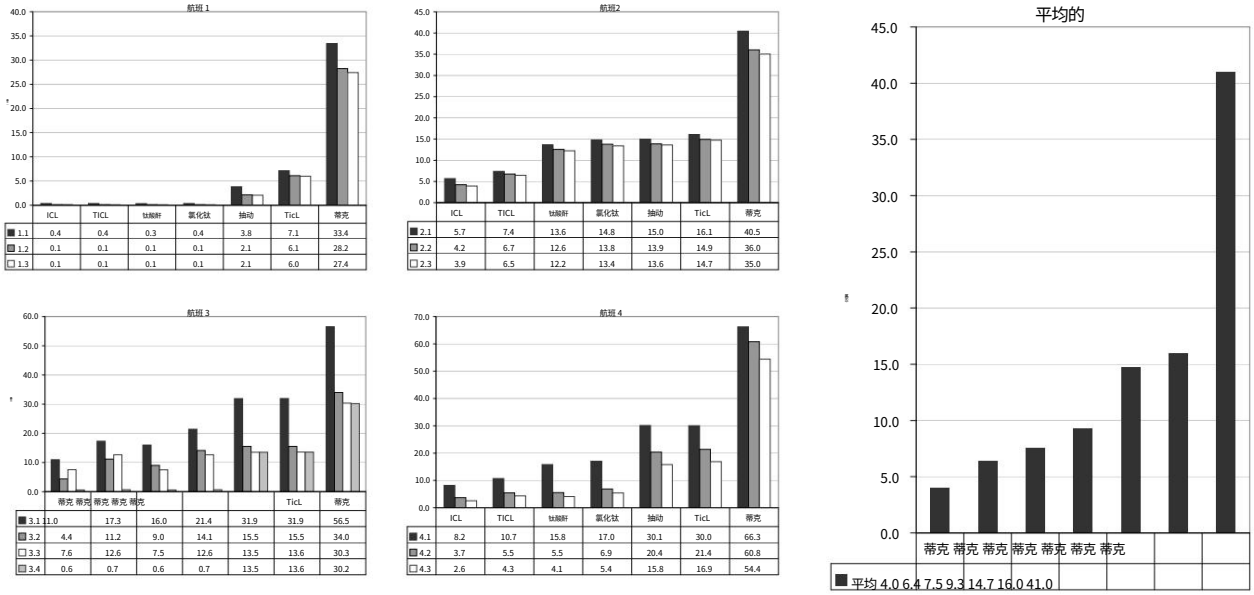


图 7:(a) 删除各种优化后按查询飞行计算的 C-Store 性能数据。四字母代码表示 C-Store配置:T=一次元组处理,t=块处理； I=启用隐形连接,i=禁用； C=压缩启用,c=禁用； L=启用延迟实现,l=禁用。 (b) 所有查询中 C-Store 的平均性能数据。

使用“getNext”的版本。我们只注意到一个显着的差异
使用此方法执行选择操作。

图 7(a) 显示了从 C 存储中连续删除这些优化的每个查询的详细结果,以及所有查询的平均值
SSBM 查询如图 7(b) 所示。块处理可以将性能提高 5% 到 50%,具体取决于压缩是否已被删除（当

取消压缩后,块处理对 CPU 的好处是
没有那么重要,因为 I/O 成为一个因素）。在其他系统中,
例如 MonetDB/X100,它们针对块处理进行了更仔细的优化 [9],如果删除此优化,人们可能会看到更大的性能下降。

隐形连接将性能提高了 50-75%。由于 C-Store 在没有不可见连接的情况下使用类似的“后期物化连接”技术,因此这种性能差异在很大程度上是

由于谓词间重写优化。有
在 SSBM 的许多情况下,可以使用谓词间重写优化。在供应商表中,地区、国家和

城市柱是粒度越来越细的属性,
如第 5.4 节所述,产生连续的位置结果
从任何这些列上的等式谓词应用程序设置。
客户表具有类似的地区、国家和城市列
三人组。零件表具有制造商、类别和品牌作为越来越细粒度的属性。最后,日期表有年、月、
以及粒度的日增加。 SSBM 中的每个查询都包含一个或多个联接（除了第一个查询航班之外的所有查询都包含多个联接）
多于一个连接）,并且对于每个查询,至少有一个连接是
具有这些特殊类型之一的谓词的维度表
的属性。因此,可以使用 Between-predicate
每个查询至少重写一次优化。

显然,最重要的两个优化是压缩
和较晚的实现。压缩通过以下方式提高性能
平均而言几乎是两倍。然而,正如第 5 节中提到的,我们不会以多重排序方式冗余存储事实表

命令以获得压缩的全部优势（只有一列 -
orderdate 列 – 已排序,另外两个列已排序
– 数量和折扣栏）。事实表中由 SSBM 查询访问的列如果没有顺序,则不太可压缩,因为它们要么是键

（具有高基数)或者是随机值。第一个查询
Flight,它访问具有顺序的三列中的每一列
它们展示了压缩的性能优势
查询访问高度可压缩的数据。在这种情况下,压缩
导致性能提高一个数量级。这
这是因为三个有序列中的值游程可以进行游程长度编码 (RLE)。游程编码不仅产生
良好的压缩比,从而减少 I/O 开销,但 RLE 是
直接操作也非常简单（例如,谓词或
聚合可以立即应用于整个运行）。首要的
排序列 orderdate 仅包含 2405 个唯一值,等等
该专栏的平均游程长度接近 25,000。该列占用的空间不到 64K。

另一个重要的优化是后期实现。这
由于需要解压缩数据,所以最后删除了优化
在元组构建过程中,以及早期物化结果
在面向行的执行中,排除了不可见的连接。晚的
实现后,性能几乎提高了三倍。这主要是因为选择性谓词

一些 SSBM 查询。谓词的选择性越强,
更浪费的是在查询计划开始时构造元组,
因为这些元组会立即被丢弃。

请注意,一旦删除所有这些优化,列存储就像行存储一样。列立即缝合在一起,完成后,处理与行存储相同。

既然是这种情况,人们会期望列存储的执行方式与面向行的物化视图案例类似:

图 5（在系统 X 和 C 存储中）,因为 I/O 要求和查询处理相似 - 唯一的区别

Machine Translated by Google

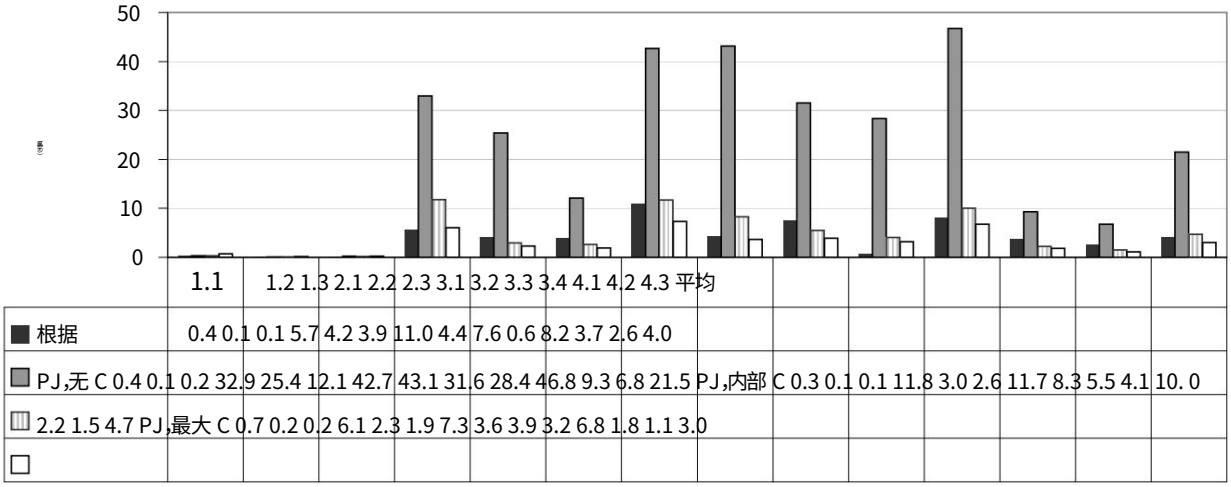


图 8:原始 SSBM 模式上的基线 C-Store 与该模式的非规范化版本的性能比较。
非规范化列要么不压缩（“PJ,No C”）,要么将字典压缩为整数（“PJ,Int C”）,要么尽可能压缩（“PJ,Max C”）。

是列存储查询计划开始时必要的元组构造。第 6.1 节警告不要与 System X 进行直接比较,但是通过将这些数字与图 5 中的“CS Row-MV”案例进行比较,我们可以看到元组构造的成本有多么昂贵（它几乎增加了 2 倍）。这与之前的结果一致[5]。

6.3.3 连接性能的影响在分析代码时,我们注意到在基线 C-

Store 情况下,性能主要在查询计划的较低部分（谓词应用程序）,并且隐形连接技术使连接性能相对便宜。为了进一步探索这一观察结果,我们创建了事实表的非规范化版本,其中事实表及其维度表是预先连接的,这样事实表就不再包含维度表中的外键,而是包含所有在尺寸表中找到的值

对每个事实表记录重复该操作（例如,所有客户信息都包含在与该客户进行的购买相对应的每个事实表元组中）。显然,从行存储的性能角度来看,这种完全的非规范化会更加有害,因为这会显着扩大表的范围。然而,在列存储中,人们可能认为这会加速只读查询,因为只有那些与查询相关的列需要读入,并且可以避免联接。

令人惊讶的是,我们发现情况往往并非如此。图 8 将上一节（使用不可见联接)的基准 C 存储性能与使用已提前执行联接的单个非规范化表的三个版本的相同基准测试中的 C 存储性能进行了比较。在第一种情况下,完整的字符串（例如客户区域和客户国家/地区)未经修改地包含在非规范化表中。此案例的性能比基本案例差 5 倍。这是因为隐形联接将维度表属性上的谓词转换为事实表外键值上的谓词。当表反规范化时,对事实表中的实际字符串属性执行谓词应用。在这两种情况下,该谓词应用都是主要步骤。但是,由于整数属性较小,因此整数外键上的谓词执行速度比字符串属性上的谓词更快。

当然,在非规范化之前,字符串属性可以很容易地被字典编码为整数。当我们这样做的时候

这样（图 8 中的“PJ,Int C”情况）,基线和非规范化情况之间的性能差异变得小得多。尽管如此,对于相当多的查询,基准情况仍然执行得更快。造成这种情况的原因是双重的。首先,某些 SSBM 查询在同一维度表上有两个谓词。

不可见连接技术能够将双谓词应用的结果总结为事实表中外键属性上的单个谓词。然而,对于非规范化的情况,谓词必须完全应用于事实表中的两列（请记住,对于数据仓库来说,事实表通常比维度表大得多,因此事实表上的谓词应用比谓词要昂贵得多尺寸表上的应用）。

其次,许多查询对维度表中的一个属性有一个谓词,并按同一维度表中的不同属性进行分组。对于不可见连接,这需要迭代外键列一次以应用谓词,并再次（在应用并交叉所有表中的所有谓词之后)以提取分组依据属性。但由于 C-Store 使用流水线执行,因此外键列中的块在第二次访问时仍将保留在内存中。在非规范化情况下,谓词列和分组依据列是事实表中的单独列,并且必须对两者进行迭代,从而使必要的 I/O 加倍。

事实上,查询中访问的许多 SSBM 维度表列的基数较低,可以压缩为小于整数外键的值。当使用完整的 C 存储压缩时,我们发现非规范化技术更有用（如图 8 中的“PJ,Max C”情况所示）。

这些结果具有有趣的含义。长期以来,非规范化一直被用作数据库系统中的一种技术,通过减少查询时必须执行的联接数量来提高查询性能。一般来说,智慧流派认为,非规范化会牺牲查询性能,从而使表更宽、更冗余（增加磁盘上表的大小并增加更新异常的风险）。人们可能会认为这种权衡在列存储中会更有利（反规范化应该更频繁地使用）,因为反规范化的缺点之一（使表更宽)在使用面向列的存储时并不存在问题布局。然而,这些结果恰恰相反:非规范化实际上并不是很有用

Machine Translated by Google

充分用于列存储（至少对于星型模式）。这是因为隐形连接的性能非常好，以至于通过非规范化减少连接数量所带来的好处微不足道。事实上，只有当事实表中包含的维度表属性已排序（或二次排序）或以其他方式高度可压缩时，反规范化才显得有用。

七、结论

在本文中，我们在数据仓库基准 SSBM 上将 C-Store 的性能与商业行存储系统的几种变体进行了比较。我们表明，尝试通过垂直分区和仅索引计划等技术来模拟行存储中列存储的物理布局不会产生良好的性能。我们将这种缓慢归因于高元组重建成本，以及狭窄的垂直分区表中每元组的高开销。我们分析了列存储能够如此有效地处理面向列的数据的原因，发现后期物化将性能提高了三倍，而压缩平均提供了大约两倍，或者说是一个数量级-访问排序数据的查询的幅度。我们还提出了一种新的连接技术，称为隐形连接，它进一步将性能提高了约 50%。

这项工作的结论并不是说在行存储中模拟列存储是不可能的。相反，这种模拟在当今的行存储系统上表现不佳（我们的实验是在 System X 的最新产品版本上进行的）。成功的面向列的模拟将需要一些重要的系统改进，例如虚拟记录 ID、减少元组开销、排序数据的快速合并连接、跨多个元组的游程编码以及一些面向列的查询执行技术，例如直接对压缩数据、块处理、不可见连接和后期物化进行操作。其中一些改进已经在各种不同的行存储中实施或提议实施[12、13、20、24]；然而，构建一个完整的行存储，可以在列存储表现良好的工作负载上转换为列存储，是一个值得追求的有趣的研究问题。

8.致谢

我们感谢 Stavros Harizopoulos 对本文的评论，并感谢 NSF 通过拨款 0704424 和 0325525 资助这项研究。

9. 重复性评估

所有包含从 C-Store 原型实验得出的数字的图（图 7a、图 7b 和图 8)均已得到 SIGMOD 可重复性委员会的验证。我们感谢 Ioana Manolescu 和可重复性委员会的反馈。

10. 参考文献

[1] http://www.sybase.com/products/ informationmanagement/sybaseiq。

[2] TPC-H 结果亮点规模 1000GB。 http://www.tpc.org/tpch/results/ tpch结果detail.asp?id=107102903。

[3]DJ阿巴迪。面向列的数据库系统中的查询执行。麻省理工学院博士论文，2008 年。博士论文。

[4] DJ Abadi、SR Madden 和 M. Ferreira。在面向列的数据库系统中集成压缩和执行。 SIGMOD，第 671-682 页，2006 年。

[5] DJ 阿巴迪、DS 迈尔斯、DJ 德威特和 SR Madden。面向列的 DBMS 中的物化策略。 ICDE，第 466-475 页，2007 年。

[6] A. Ailamaki、DJ DeWitt、MD Hill 和 M. Skounakis。编织关系以提高缓存性能。 VLDB，第 169-180 页，2001 年。

[7] DS 巴托里。关于搜索转置文件。 ACM 翻译。数据库系统，4(4)：531–544，1979。

[8] PA Bernstein 和 D.-MW Chiu。使用半连接来解决关系查询。 J.ACM，28(1)：25–40，1981。

[9] P. Boncz、M. Zukowski 和 N. Nes。 MonetDB/X100：超流水线查询执行。 CIDR，2005 年。

[10] PA Boncz 和 ML Kersten。用于查询支离破碎的世界的 MIL 原语。 VLDB 杂志，8(2)：101–119，1999。

[11] G.格雷夫。 Volcano - 一个可扩展的并行查询评估系统。 6：120–135，1994 年。

[12] G.格雷夫。 B 树中的高效列式存储。 SIGMOD 建议，36(1):3–6，2007。

[13] A. Halverson、JL Beckmann、JF Naughton 和 DJ 德威特。通用框架中 C 存储和行存储的比较。技术报告 TR1570，威斯康星大学麦迪逊分校，2006 年。

[14] S. Harizopoulos、V. Liang、DJ Abadi 和 SR Madden。读取优化数据库中的性能权衡。 VLDB，第 487-498 页，2006 年。

[15] S. Harizopoulos、V. Shkapenyuk 和 A. Ailamaki。 QPipe：同步管道关系查询引擎。 SIGMOD，第 383-394 页，2005 年。

[16] S. Khoshafian、G. Copeland、T. Jagodis、H. Boral 和 P. Valduriez。分解存储模型的查询处理策略。 ICDE，第 636-643 页，1987 年。

[17] P.奥尼尔和G.格雷夫。多表连接通过位图连接索引。 SIGMOD 建议，24(3):8–11，1995。

[18] PE O Neil，X. Chen，EJ O Neil。相邻维度列索引（ADC 索引)可提高星型模式查询性能。在 ICDE，2008 年。

[19] PE O Neil，EJ O Neil，X. Chen。星型模式基准 (SSB)。 http:// www.cs.umb.edu/ poneil/ StarSchemaB.PDF 。

[20] S. Padmanabhan、T. Malkemus、R. Agarwal 和 A.金兰。现代计算机体系结构中关系数据库操作的面向块处理。在 ICDE，2001 年。

[21] R. Ramamurthy、D. Dewitt 和 Q. Su。破碎镜子的箱子。 VLDB，第 89 – 101 页，2002 年。

[22] M. Stonebraker、DJ Abadi、A. Batkin、X. Chen，M. Cherniack、M. Ferreira、E. Lau、A. Lin、SR Madden、EJ O Neil、PE O Neil、A. Rasin、N. Tran 和 SB 兹多尼克。 C-Store：面向列的 DBMS。 VLDB，第 553-564 页，2005 年。

[23] A.韦宁格。在星型模式中高效执行联接。 SIGMOD，第 542-545 页，2002 年。

[24] J.周和KA罗斯。缓冲数据库操作增强的指令缓存性能。 SIGMOD，第 191-202 页，2004 年。

[25] M. Zukowski、PA Boncz、N. Nes 和 S. Heman。MonetDB/X100 - CPU 缓存中的 DBMS。 IEEE 数据工程公告，28(2):17–22，2005 年 6 月。

[26] M. Zukowski、S. Heman、N. Nes 和 P. Boncz。超标量 RAM-CPU 高速缓存压缩。在 ICDE，2006 年。